

Help Menu

The commands are described under the [Help Menu](#).

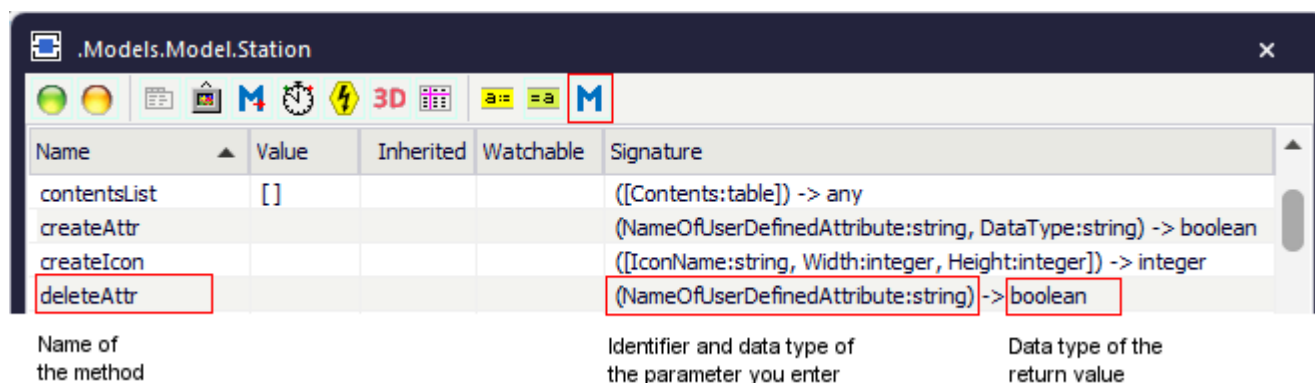
Methods of the Dialog

_Methods of the Dialog

The *Dialog* provides:

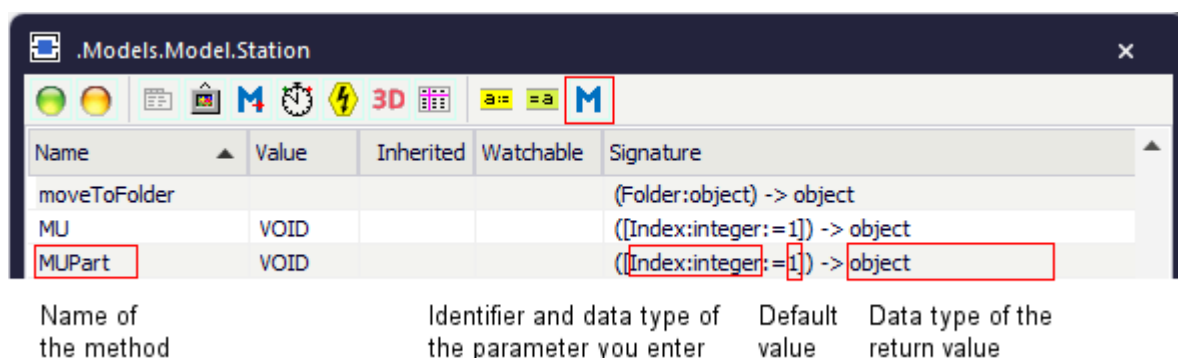
- The *methods* listed in the table of contents to the left.
- The [Methods of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.



Name	Value	Inherited	Watchable	Signature
contentsList	[]			((Contents:table)) -> any
createAttr				(NameOfUserDefinedAttribute:string, DataType:string) -> boolean
createIcon				(IconName:string, Width:integer, Height:integer) -> integer
deleteAttr				(NameOfUserDefinedAttribute:string) -> boolean

Name of the method Identifier and data type of the parameter you enter Data type of the return value



Name	Value	Inherited	Watchable	Signature
moveToFolder				(Folder:object) -> object
MU	VOID			((Index:integer:=1)) -> object
MUPart	VOID			((Index:integer:=1)) -> object

Name of the method Identifier and data type of the parameter you enter Default value Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, *:= false* in the example above.

If the *method* has a return value, the signature shows its data type after the arrow *->*, *→ boolean* in the example above.

clearData [SimTalk]

Deletes the dialog items of the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.clearData
```

Example

```
MyDialog.clearData
```

close [SimTalk] - Dialog

Closes the dialog box of the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.close(OK:boolean)
```

Parameter

The parameter *OK* of data type *boolean* designates the action which the dialog takes, **OK** or **Cancel**.

Specify `true` to apply the changes the user made.

Specify `false` to cancel all changes.

Example

```
MyDialog.close(true)
```

SimTalk

[open \[SimTalk\] - Dialog](#)

See also

[Show Dialog \[Dialog\]](#)

closeDialog [SimTalk] - Dialog

Closes the dialog box of the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.closeDialog([ApplyChanges:boolean:=true]) → boolean
```

Parameter

The optional parameter *ApplyChanges* of data type *boolean* determines if *Plant Simulation* ignores all changes the user made in the dialog or if it accepts and applies these changes.

Specify `true` to make the *Dialog* evaluate all settings in the dialog box and apply them as the new values.

Specify `false` to make the *Dialog* discards the changes.

Default Value of the Parameter

The default value is `true`.

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.closeDialog(true)
```

SimTalk

[openDialog \[SimTalk\] - Dialog](#)

See also

[Show Dialog \[Dialog\]](#)

createButton [SimTalk]

Creates a *Button* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createButton(Name:string, X:integer, Y:integer, Width:integer[,  
Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the button.

Note

You can set the **Caption** of the button with the method **setCaption**.

- The parameter *X* of data type *integer* designates the position of the button on the **X-axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the button on the **Y-axis** in the *Dialog*.
- The parameter *Width* of data type *integer* designates the **Width** of the button.
- The optional parameter *Parent* of data type *string* designates the parent of the drop-down list box, for example a **Group Box** or **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MyTabbedDialog.createButton("Order", 7, 7, 18, "MyTab4")
```

See also

New Button

createCheckBox [SimTalk]

Creates a *Check Box* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createCheckBox(Name:string, X:integer, Y:integer[, Parent:string])
→ boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the check box.

Note

You can set the **Caption** of the check box with the method **setCaption**.

- The parameter *X* of data type *integer* designates the position of the check box on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the check box on the **Y-Axis** in the *Dialog*.
- The optional parameter *Parent* of data type *string* designates the parent of the check box, for example a **Group Box** or **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MySimpleDialog.createCheckBox("SunRoof", 9, 6)
```

SimTalk

[getValue \[SimTalk\] - Dialog](#)

See also

[New Check Box](#)

createDropDownListBox [SimTalk]

Creates a *Drop-Down List Box* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createDropDownListBox(Name:string, X:integer, Y:integer,  
Width:integer[, Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the drop down list box.
- The parameter *X* of data type *integer* designates the position of the drop- down list box on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the drop- down list box on the **Y-Axis** in the *Dialog*.
- The parameter *Width* of data type *integer* designates the **Width** of the drop down list box.
- The optional parameter *Parent* of data type *string* designates the parent of the drop-down list box, for example a **Group Box** or **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.createDropDownListBox("unit", 12, 1, 10, "Page.Group")  
units.create  
units.insert(1,"mm")  
units.insert(2,"m")  
units.insert(3,"km")  
MyDialog.setList("unit",units)
```

SimTalk

[getValue \[SimTalk\] - Dialog](#)

See also

[New DropDownList Box](#)

createEditTextBox [SimTalk]

Creates a *Edit Text Box* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createEditTextBox(Name:string, X:integer, Y:integer, Width:integer[, Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the edit text box.
- The parameter *X* of data type *integer* designates the position of the text box on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the text box on the **Y-Axis** in the *Dialog*.
- The parameter *Width* of data type *integer* designates the **Width** of the edit text box.
- The optional parameter *Parent* of data type *string* designates the parent of the edit text box, for example a **Group Box** or **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MySimpleDialog.createEditTextBox("Vanity text", 15, 7, 18)
```

SimTalk

[getValue \[SimTalk\] - Dialog](#)

See also

[New Edit Text Box](#)

createGroupBox [SimTalk]

Creates a *Group Box* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createGroupBox(Name:string, X:integer, Y:integer, Width:integer, Height:integer[, Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the group box.

Note

You can set the **Caption** of the group box with the method **setCaption**.

- The parameter *X* of data type *integer* designates the position of the group box on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the group box on the **Y-Axis** in the *Dialog*.
- The parameter *Width* of data type *integer* designates the **Width** of the group box.
- The parameter *Height* of data type *integer* designates the **Height** of the group box.
- The optional parameter *Parent* of data type *string* designates the parent of the group box, for example a **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MySimpleDialog.createGroupBox("Drive", 4, 26, 4)
```

See also

New Group Box

createImage [SimTalk]

Creates an *Image* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createImage(Name:string, X:integer, Y:integer[, Parent:string]) →  
boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the image.
- The parameter *X* of data type *integer* designates the position of the image on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the image on the **Y-Axis** axis in the *Dialog*.
- The optional parameter *Parent* of data type *string* designates the parent of the image, for example a **Group Box** or a **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MyTabbedDialog.createImage("ImageItem", 40, 1, "MyTab4")  
MyTabbedDialog.setIcon("ImageItem", "CompanyLogo")
```

SimTalk

[setIcon \[SimTalk\]](#)

See also

[New Image](#)

createListBox [SimTalk]

Creates a *List Box* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createListBox(Name:string, X:integer, Y:integer, Width:integer, Height:integer[, Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the list box.
- The parameter *X* of data type *integer* designates the position of the list box on the **X-axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the list box on the **Y-axis** in the *Dialog*.
- The parameter *Width* of data type *integer* designates the **Width** of the list box.
- The parameter *Height* of data type *integer* designates the **Height** of the list box.
- The optional parameter *Parent* of data type *string* designates the parent of the list box, for example a **Group Box** or a **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MyTabbedDialog.createListBox("ModelList", 18, 0, 16, 2, "MyTab1")
```

SimTalk

[getValue \[SimTalk\] - Dialog](#)

See also

[New List Box](#)

createListView [SimTalk]

Creates a *List View* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createListView(Name:string, X:integer, Y:integer, Width:integer, Height:integer[, Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the list view.
- The parameter *X* of data type *integer* designates the position of the list view on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the list view on the **Y-Axis** in the *Dialog*.
- The parameter *Width* of data type *integer* designates the **Width** of the list view.
- The parameter *Height* of data type *integer* designates the **Height** of the list view.
- The optional parameter *Parent* of data type *string* designates the parent of the list view, for example a **Group Box** or a **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MyTabbedDialog.createListView("Configuration", 0, 1, 36, 5, "MyTab4")
```

See also

[New List View](#)

createMenu [SimTalk]

Creates a *Menu/Menu Command* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createMenu(Name:string[, Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the menu.

Note

You can set the **Caption** of the menu with the method **setCaption**.

- The optional parameter *Parent* of data type *string* designates the parent of the menu items or of the sub-menus. For the top level menu you do not need a parent.

Return Value

The return value has the data type *boolean*.

Example

```
MySimpleDialog.createMenu("Show")
```

See also

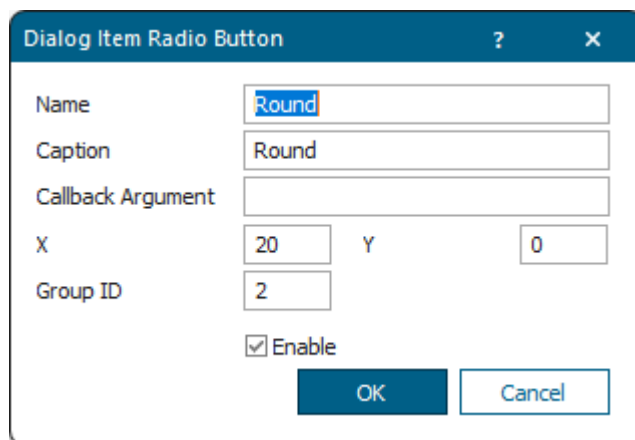
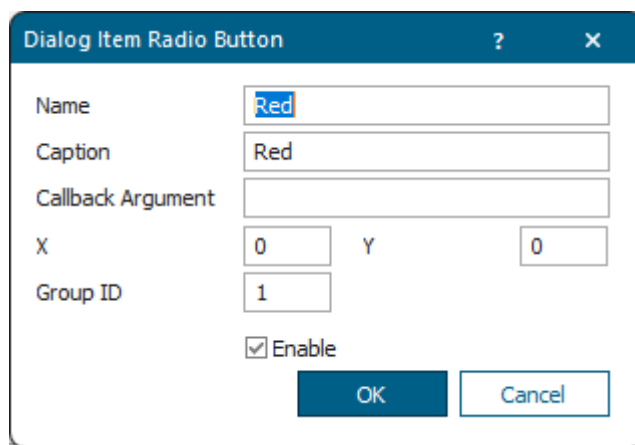
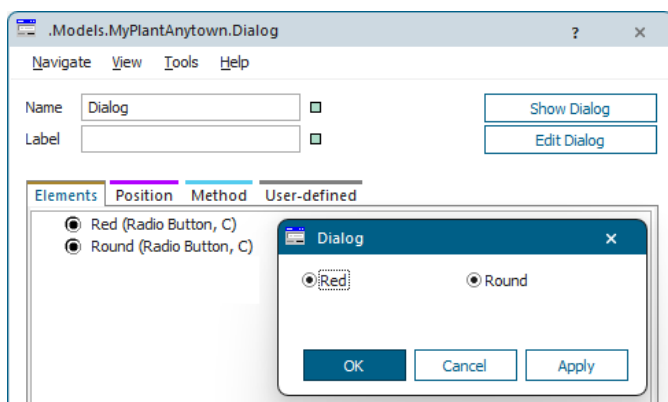
[New Menu/New Menu Command](#)

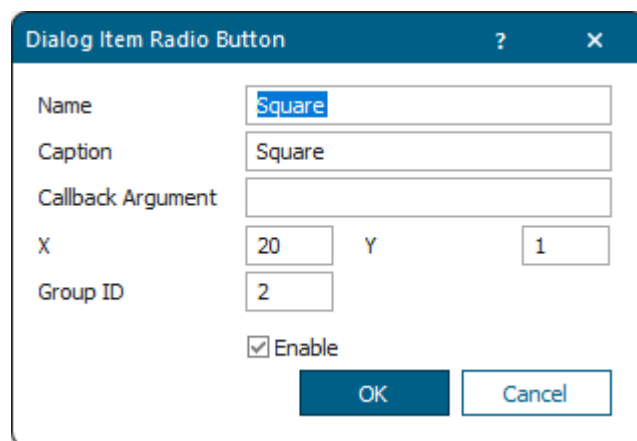
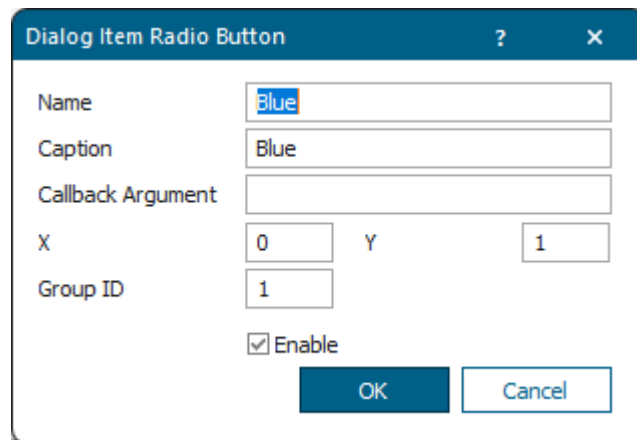
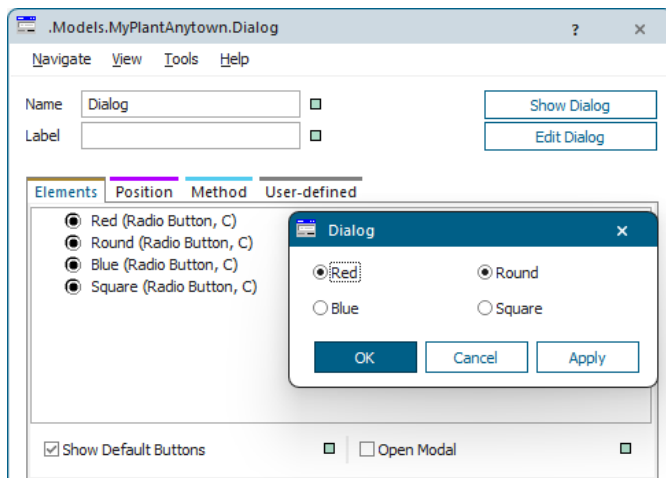
createRadioButton [SimTalk]

Creates a *Radio Button* in the *Dialog* designated by <Path>.

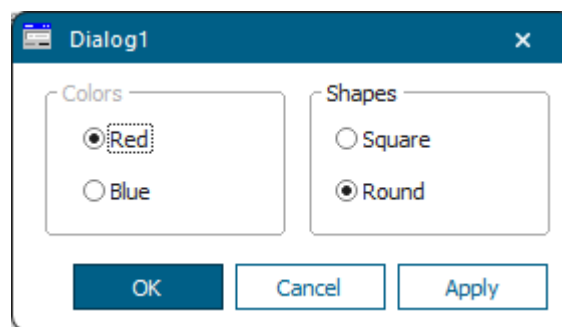
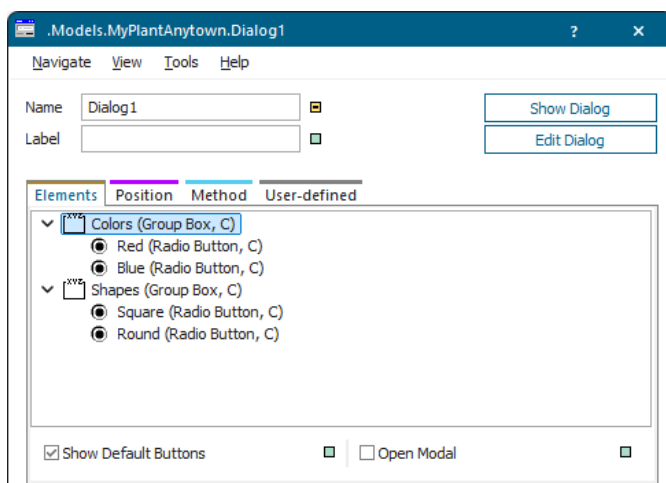
Remarks

You have to create all items of a group of radio buttons sequentially because *Plant Simulation* creates the items according to their row and column position. You cannot arrange two sets of radio buttons as shown below, where the radio buttons **Red** and **Blue** on the left-hand side and the radio buttons **Round** and **Square** on the right-hand side form a group each. Windows ignores the **Group ID** you enter and groups the radio buttons row by row according to the **Y**-coordinate, i.e., it treats **Red** and **Round** as one group and **Blue** and **Square** as the other group.





If you want to model like this, do not place the radio buttons loosely onto the dialog, but group them within group boxes.



Type

Method

Syntax

```
<Path>.createRadioButton(Name:string, X:integer, Y:integer[,  
Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the radio button.

Note

You can set the **Caption** of the radio button with the method [setCaption](#).

- The parameter *X* of data type *integer* designates the position of the radio button on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the radio button on the **Y-Axis** in the *Dialog*.
- The optional parameter *Parent* of data type *string* designates the parent of the radio button, for example a **Group Box** or a **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MySimpleDialog.createRadioButton("Frontwheel", 3, 0)
```

SimTalk

[setGroupID \[SimTalk\]](#)

[getValue \[SimTalk\] - Dialog](#)

See also

[New Radio Button](#)

[Add a Set of Radio Buttons](#)

createStaticTextBox [SimTalk]

Creates a *Static Text Box* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createStaticTextBox(Name:string, X:integer, Y:integer[,  
Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the static text box.

Note

You can set the **Caption** of the static text box with the method **setCaption**.

- The parameter *X* of data type *integer* designates the position of the static text box on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the static text box on the **Y-Axis** in the *Dialog*.
- The optional parameter *Parent* of data type *string* designates the parent of the static text box, for example a **Group Box** or a **Tab Page**.

Return Value

The return value has the data type *boolean*.

Example

```
MySimpleDialog.createStaticTextBox("Vanity", 5, 7)
```

See also

[New Static Text Box](#)

createTabControl [SimTalk]

Creates a *Tab Control* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createTabControl(Name:string, X:integer, Y:integer, Width:integer, Height:integer) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the tab control.
- The parameter *X* of data type *integer* designates the position of the tab control on the **X-Axis** in the *Dialog*.
- The parameter *Y* of data type *integer* designates the position of the tab control on the **Y-Axis** in the *Dialog*.
- The parameter *Width* of data type *integer* designates the **Width** of the tab control.
- The parameter *Height* of data type *integer* designates the **Height** of the tab control.

Return Value

The return value has the data type *boolean*.

Example

```
MyTabbedDialog.createTabControl("TabControl", 0, 0, 64, 10)
```

SimTalk

[getValue \[SimTalk\] - Dialog](#)

See also

[New Tab Control](#)

createTabPage [SimTalk]

Creates a *Tab Page* in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.createTabPage(Name:string[, Parent:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **Name** of the tab page.

Note

You can set the **Caption** of the tab page with the method **setCaption**.

- The optional parameter *Parent* of data type *string* designates the parent of the tab page, i.e., the **Tab Control**.

Return Value

The return value has the data type *boolean*.

Example

```
MyTabbedDialog.createTabPage("MyTab1")
```

See also

New Tab Page

deleteItem [SimTalk]

Deletes the specified dialog item from the *Dialog* specified by <Path>.

Type

Method

Syntax

```
<Path>.deleteItem(DialogItem:string) → boolean
```

Parameters

The parameter *DialogItem* of data type *string* designates the dialog item, which you would like to delete.

Return Value

The return value has the data type *boolean*.

true if the dialog items was deleted sucessfully.

false if the dialog item was not deleted because the dialog item is inherited for example.

Example

```
MyTabbedDialog.deleteItem("SunRoof")
```

See also

[New Tab Page](#)

getCheckBox [SimTalk]

Returns if the **Check Box** or the **Radio Button** in the *Dialog* designated by <Path> is selected (true) or cleared (false).

Type

Method

Syntax

```
<Path>.getCheckBox(DialogItem:string) → boolean
```

Parameter

The parameter *DialogItem* of data type *string* designates the **Check Box** or the **Radio Button**.

Return Value

The return value has the data type *boolean*.

Example

```
print MyDialog.getCheckBox("My_CheckBox")
```

See also

[setCheckBox \[SimTalk\]](#)

getIcon [SimTalk]

Returns the **Image ID** or the image name of the image in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.getIcon(DialogItem:string) → string
```

Parameter

The parameter *DialogItem* of data type *string* designates the dialog item.

Return Value

The return value has the data type *string*.

Example

```
print MyDialog.getIcon("My_Image")
```

SimTalk

[setIcon \[SimTalk\]](#)

See also

[New Image](#)

[Image ID](#)

getIndex [SimTalk]

Returns the number of the designated dialog item in the *Dialog* designated by <Path>.

Type

Method

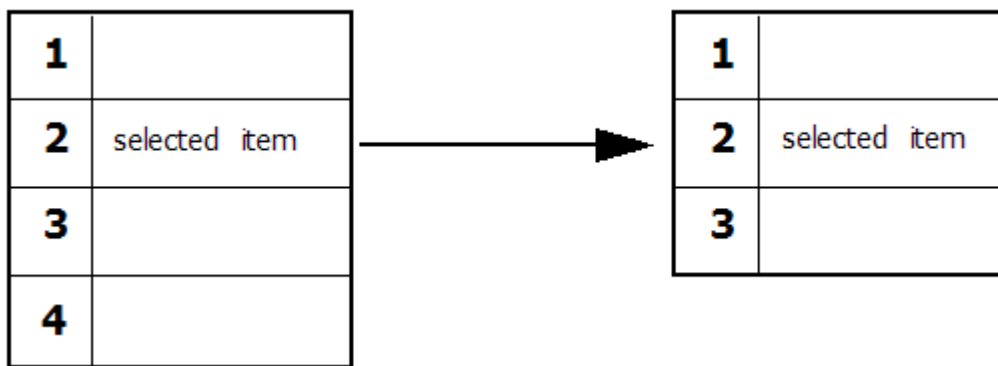
Syntax

```
<Path>.getIndex(DialogItem:string) → integer
```

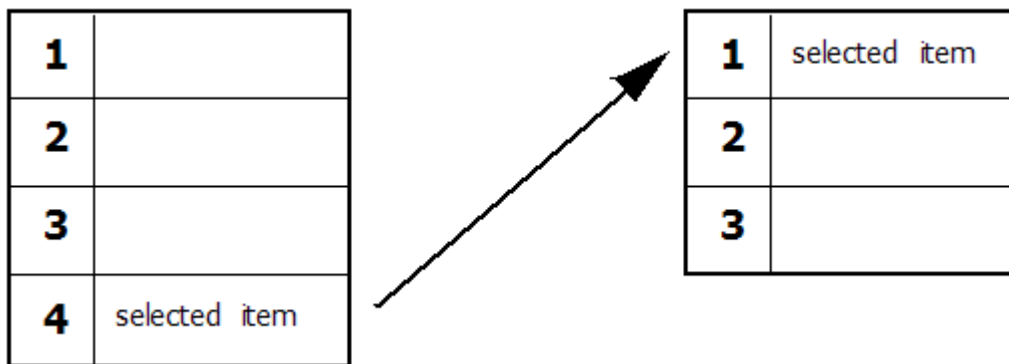
Parameter

The parameter *DialogItem* of data type *string* designates the **List Box**, the **Drop-Down List Box**, or the **Tab Page**. The leftmost tab has the number 1.

If you assign another list, and if it is possible, the *Dialog* selects the item at the same position in the list box.



If you assign another list, and it is not possible to select the same position in the list box that the previous selection had, the *Dialog* automatically selects the first item in the new list.



Return Value

The return value has the data type *integer*.

Is 0 if the drop-down list box does not contain any items.

Is 1 if it does contain items, and no item is selected. Then the *Dialog* automatically selects the first item in the list.

Example

```
print MyDialog.getIndex("doors")
print MyDialog.getIndex("units")
```

SimTalk

[setIndex \[SimTalk\]](#)

See also

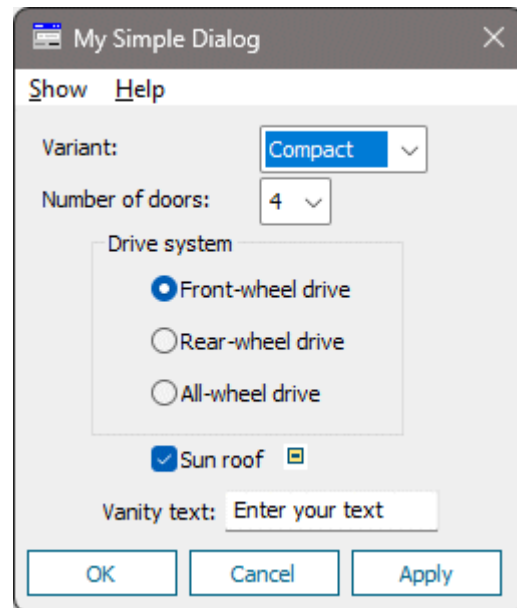
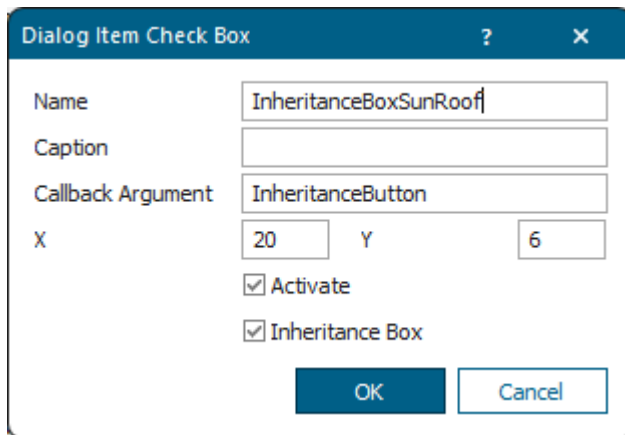
Position of the item in the [Items \[Dialog\]](#) list

getInheritanceBox [SimTalk]

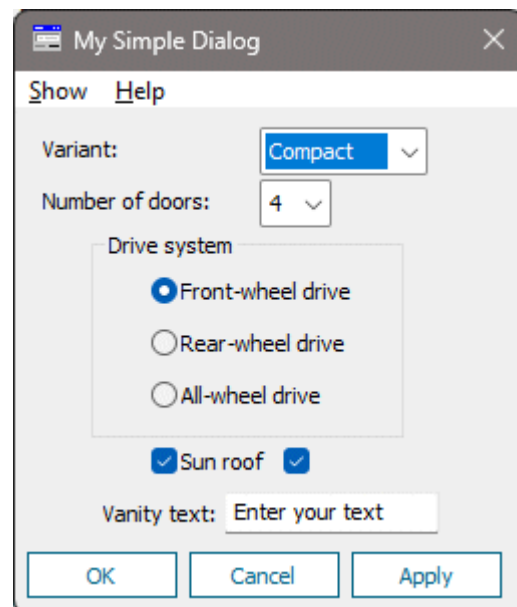
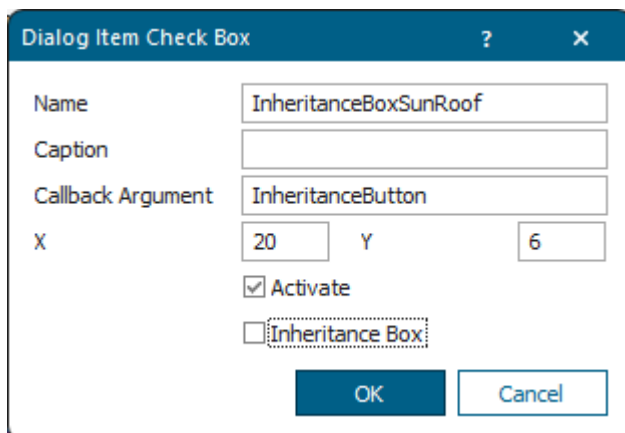
Returns the current mode (selected or cleared) of the **Inheritance Box** of the specified check box in the *Dialog* designated by <Path>.

Remarks

If you assigned `true`, the *Dialog* shows the check box as a check box for inheritance.



If you assigned `false`, the *Dialog* shows the check box as a normal check box.



Type

Method

Syntax

```
<Path>.getInheritanceBox(DialogItemName:string) → boolean
```

Parameter

The parameter *DialogItemName* of data type *string* designates the name of the check box which you defined as an **Inheritance Box**.

Return Value

The return value has the data type *boolean*.

Example

```
print MySimpleDialog.getInheritanceBox("InheritanceBoxSunRoof")
```

SimTalk

[setInheritanceBox \[SimTalk\]](#)

See also

[New Check Box](#)

getItemsList [SimTalk]

Returns the full path within the dialog, the **name** and the type of dialog item in the *Dialog* designated by <Path> and writes it into a table.

Type

Method

Syntax

```
<Path>.getItemsList(Table:table)
```

Parameter

The parameter *Table* of data type *table* designates the name of the table.

.Models.MyPlantAnytown.MyElementsTable					
TabControl					
	string 1	string 2	string 3	string 4	string 5
1	TabControl	TabControl	Tab Control		CBTabControl
2	TabControl.MyTab1	MyTab1	Tab	Variant	
3	TabControl.MyTab1.Model	Model	Static Text Box	Choose the model:	
4	TabControl.MyTab1.ModelList	ModelList	List Box	Cruiser;Racer;Family;Van;Custom;Bigwheel;Hot rod;	CBModelList
5	TabControl.MyTab1.Variant	Variant	Static Text Box	Choose the variant:	
6	TabControl.MyTab1.VariantList	VariantList	List Box	Station wagon;Convertible;Targa;All-terrain;	CBVariantList
7	TabControl.MyTab2	MyTab2	Tab	Accessories	
8	TabControl.MyTab3	MyTab3	Tab	Custom Add-ons	
9	TabControl.MyTab4	MyTab4	Tab	Your Configuration	
10	TabControl.MyTab4.CustomerSelection	CustomerSelection	Static Text Box	You have selected these features:	
11	TabControl.MyTab4.Configuration	Configuration	List View		CBConfiguration
12	TabControl.MyTab4.OrderedItem	OrderedItem	Image		
13	TabControl.MyTab4.Order	Order	Button	Submit Order	CBOrder
14	NewTabControl	NewTabControl	Tab Control		

Example

```
MyDialog.getItemsList(MyElementsTable)
```

See also

[Name \[dialog item in Dialog\]](#)

getPasswordMasking [SimTalk]

Returns the state of the check box **Password** for a dialog item of type **Edit Text Box** in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.getPasswordMasking(DialogItem:string) → boolean
```

Parameter

The parameter *DialogItem* of data type *string* designates the check box **Password**.

Return Value

The return value has the data type *boolean*.

true if the check box is selected.

false if the check box is cleared.

Example

```
print MyDialog.getPasswordMasking("MyEditTextBox")
```

SimTalk

[setPasswordMasking \[SimTalk\]](#)

See also

[New Edit Text Box](#)

getTable [SimTalk] - Dialog

Returns the name of the *table* that defines the items which a **List View** in the *Dialog* designated by <Path> shows.

Type

Method

Syntax

```
<Path>.getTable(ListView:string) → string
```

Parameter

The parameter *ListView* of data type *string* designates the list view.

Return Value

The return value has the data type *string*.

Example

```
var table_path : string  
table_path := MyDialog.getTable("myListView")
```

SimTalk

[setTable \[SimTalk\] - Dialog](#)

See also

[New List View](#)

getTableRow [SimTalk]

Returns the number of the row in the **Items Table** that defines the items which a **List View** in the *Dialog* designated by <Path> shows and which the user selected.

Type

Method

Syntax

```
<Path>.getTableRow(Table:string) → integer
```

Parameter

The parameter *ListView* of data type *string* designates the list view.

Return Value

The return value has the data type *integer*.

Example

```
print MyDialog.getTableRow("ListView")
```

SimTalk

[setTableRow \[SimTalk\]](#)

See also

[setTableRow \[SimTalk\]](#)

[Items \[Dialog\]](#)

[New List View](#)

getUserDialogXYWH [SimTalk]

Returns the position of the *Dialog* designated by <Path> and assigns the coordinates to the local variables, which you specify.

Type**Method****Syntax**

```
<Path>.getUserDialogXYWH(byRef X:integer, byRef Y:integer, byRef
Width:integer, byRef Height:integer)
```

Parameters

You can specify the following local variables:

- The local variable *X* of data type *integer* designates the x-coordinate of the dialog of the object.
- The local variable *Y* of data type *integer* designates the y-coordinate of the dialog of the object.
- The local variable *Width* of data type *integer* designates its width.
- The local variable *Height* of data type *integer* designates its height.

Example

```
var x,y,w,h: integer
MyDialog.getUserDialogXYWH(x,y,w,h)
print x," ",y," ",w," ",h
```

SimTalk

[setXYWH \[SimTalk\]](#)

[getXYWH \[SimTalk\]](#)

getValue [SimTalk] - Dialog

Returns the changed value of the dialog item in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.getValue(DialogItem:string) → string
```

Parameter

The parameter *DialogItem* of data type *string* designates the dialog item of type **Drop-Down List Box**, **Check Box**, **Radio Button**, **List Box**, **Edit Text Box**, and **Tab Control**. If you apply the method to the **Tab Control**, *Plant Simulation* returns the name of the active tab.

Return Value

The return value has the data type *string*.

Example

```
var value: string
var number : integer
value := MyDialog.getValue("unit")
number := str_to_num(value)
```

SimTalk

[createCheckBox \[SimTalk\]](#)

[createDropDownListBox \[SimTalk\]](#)

[createEditTextBox \[SimTalk\]](#)

[Mutation \[SimTalk\] - GASequence](#)

[createRadioButton \[SimTalk\]](#)

[createTabControl \[SimTalk\]](#)

[setCaption \[SimTalk\]](#)

open [SimTalk] - Dialog

Opens the dialog box of the *Dialog* designated by <Path> at the specified position on screen.

Type

Method

Syntax

```
<Path>.open(X:integer, Y:integer)
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the **X-Position** at which *Plant Simulation* opens the dialog box.
- The parameter *Y* of data type *integer* designates the **Y-Position** at which *Plant Simulation* opens the dialog box.

Example

```
MyDialog.open(100,100)
```

SimTalk

[close \[SimTalk\] - Dialog](#)

See also

[Show Dialog \[Dialog\]](#)

[X-Position \[dialog item\]](#)

[Y-Position \[dialog item\]](#)

openDialog [SimTalk] - Dialog

Opens the dialog box of the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.openDialog → boolean  
<Path>.openDialog(CallOpenControl:boolean:=false) → boolean
```

Parameter

The parameter *CallOpenControl* of data type *boolean* sets if *Plant Simulation* opens the dialog box the same way as double-clicking the icon does and also executes any existing **Open Control** (*true*) or if it only opens the dialog (*false*).

Default Value of the Parameter

The default value is *false*.

Return Value

The return value has the data type *boolean*.

true if *Plant Simulation* succeeded in opening the dialog.

Example

```
myDialog.openDialog  
.Models.Model.myDialog.openDialog(false)
```

SimTalk

[closeDialog \[SimTalk\] - Dialog](#)

See also

[Show Dialog \[Dialog\]](#)

[Open Control](#)

setActiveTabPage [SimTalk]

Activates the designated tab of the designated tab control of the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.setActiveTabPage(TabControl:string, TabPageName:string)
```

Parameters

You can specify the following parameters:

- The parameter *TabControl* of data type *string* designates the tab control that contains the tab to be activated.
- The parameter *TabPageName* of data type *string* designates the tab that will be activated.

Example

```
MyDialog.setActiveTabPage("MyTabControl", "Page1")
```

See also

[New Tab Control](#)

[New Tab Page](#)

setCallbackArgument [SimTalk]

Sets the **Callback Argument** which the **Callback Method** of the *Dialog* designated by <Path> executed.

Remarks

The *Console* shows this parameter if you show it in the *Callback method*.

Type

Method

Syntax

```
<Path>.setCallbackArgument(DialogItem:string, Argument:string) → integer
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the dialog item to which the callback argument applies.
- The parameter *Argument* of data type *string* designates the value of the callback argument.

Return Value

The return value has the data type *integer*.

Example

```
MyDialog.setCallbackArgument("TabControl.Page1.Save", "Bill")
```

See also

[Callback Argument \[text box\] - Dialog](#)

[Callback Method \[text box\] - Dialog](#)

setCaption [SimTalk]

Sets the **Caption**, i.e., the description of a dialog item, and the settings described below of the *Dialog* designated by <Path>.

Remarks

Plant Simulation shows changes immediately.

setCaption applies to these dialog items:

- **Static Text Box** which changes its caption.
- **Edit Text Box** which changes the contents of the text box.
- **Tab Page** which changes the label of the tab.
- **Button** which changes the label of the button.
- **Group Box** which changes its caption.
- **Menu/Menu Command** which changes its caption.
- **Check Box** which changes its caption.
- **Radio Button** which changes its caption.

Type

Method

Syntax

```
<Path>.setCaption(DialogItem:string, Caption:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the dialog item to which the caption applies.
- The parameter *Caption* of data type *string* designates the text that you want to assign as the caption.

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.setCaption("object",current.name)
if Station.failed
  MyDialog.setCaption("state","failed")
else
  MyDialog.setCaption("state","ready")
end
```

SimTalk

[Caption \[text box\] - Dialog](#)

[getValue \[SimTalk\] - Dialog](#)

setCheckBox [SimTalk]

Sets if the **Check Box** or the **Radio Button** of the *Dialog* designated by <Path> will be selected (`true`) or cleared (`false`).

Type

Method

Syntax

```
<Path>.setCheckBox(DialogItem:string, Active:boolean) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the dialog item.
- The parameter *Active* of data type *boolean* sets if it will be activated (`true`) or not (`false`).

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.setCheckBox("My_CheckBox", true)
```

SimTalk

[getCheckBox \[SimTalk\]](#)

See also

[New Check Box](#)

[New Radio Button](#)

setEditType [SimTalk]

Sets the data type of the **Edit Text Box** of the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.setEditType(DialogItem:string, Type:string)
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the name or the name and the path of the edit text box.
- The parameter *Type* of data type *string* designates which kind of characters the user can type into the edit text box.

Setting	The user can type in
Any Character	any character, which includes special characters, lower and upper case letters and numbers.
Letters and Digits	lower and upper case letters, including special characters, such as umlauts, and letters of any kind of language, underscores, and blank spaces. The user can also type in the digits 0 to 9.
Letters	lower and upper case letters, including special characters, such as umlauts, and letters of any kind of language, and blank spaces.
Decimal Number	0123456789
Signed Decimal Number	decimal numbers with a negative or positive sign: -+0123456789
Hexadecimal Number	ADbf09, for example.
Octal Number	01234567, but not 18 for example.
Binary Number	0 or 1
Floating Point Number	12.3E-45, for example.
Time	time values.
Date with Time	a date containing time values.

Setting	The user can type in
Date	a date.
Positive Real Number	a positive real number.

Example

```
MyDialog.setEditType("MyEditTextBox", "Letters and Digits")
```

See also

[New Edit Text Box](#)

setGroupID [SimTalk]

Sets the **Group ID** of dialog items of a **Radio Button** of the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.setGroupID(DialogItem:string, Group:integer)
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the dialog item.
- The parameter *Group* of data type *integer* designates the group ID.

Example

```
MySimpleDialog.setGroupID("Frontwheel", 1)  
MySimpleDialog.setGroupID("Rearwheel", 1)  
MySimpleDialog.setGroupID("MagRim", 2)  
MySimpleDialog.setGroupID("SteelRim", 2)
```

See also

[Group ID](#)

[New Radio Button](#)

setIcon [SimTalk]

Sets the **Image ID** of a dialog item of the *Dialog* designated by <Path> to the specified value.

Type

Method

Syntax

```
<Path>.setIcon(DialogItem:string, ImageID:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the dialog item,
- The parameter *ImageID* of data type *string* designates the value. You can enter an **Image ID** or an **Image Name** that is an icon of the *Dialog* to which the image belongs.

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.setIcon("My_Image", "Logo")
MyDialog.setIcon("My_Image", "3")
```

SimTalk

[getIcon \[SimTalk\]](#)[createImage \[SimTalk\]](#)

See also

[Image ID](#)

setIndex [SimTalk]

Sets the number of the item, which the **List Box**, the **Drop-down List Box**, or the **Tab Page** of the *Dialog* designated by <Path> shows.

Type

Method

Syntax

```
<Path>.setIndex(DialogItem:string, ItemNumber:integer) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the **List Box**, the **Drop-down List Box**, or the **Tab Page**.
- The parameter *ItemNumber* of data type *integer* designates the item that will be shown.

If you do not enter an item, the *Dialog* selects the first entry in the list.

The parameter *DialogItem* of data type *string* can also designate a **Tab Control**.

The parameter *ItemNumber* of data type *integer* can also designate a tab, which the dialog activates within this **Tab Control**. The leftmost tab has the number 1.

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.setIndex("units",6)  
MyDialog.setIndex("myTabControl",3)  
MyDialog.setIndex("listView",3)
```

SimTalk

[getIndex \[SimTalk\]](#)

See also

Position of the item in the [Items \[Dialog\]](#) list

[New List Box](#)

[New DropDownList Box](#)

[New Tab Page](#)

setInheritanceBox [SimTalk]

Selects (true) the **Inheritance Box** of the specified check box in the *Dialog* designated by <Path> or clears it (false).

Type

Method

Syntax

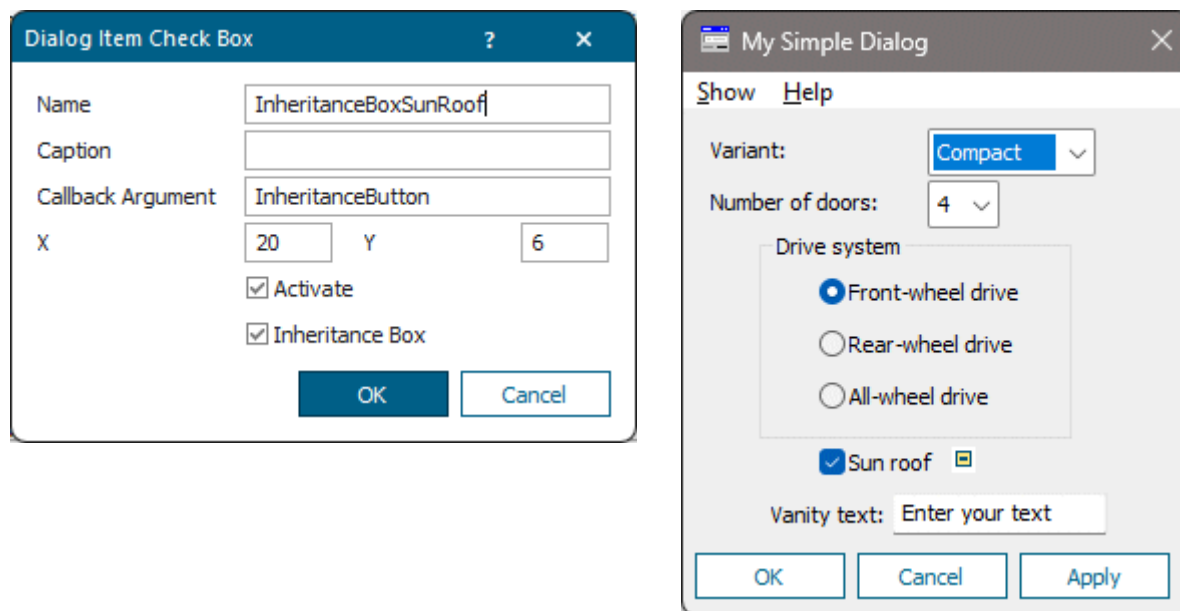
```
<Path>.setInheritanceBox(DialogItemName:string, Value:boolean)
```

Parameters

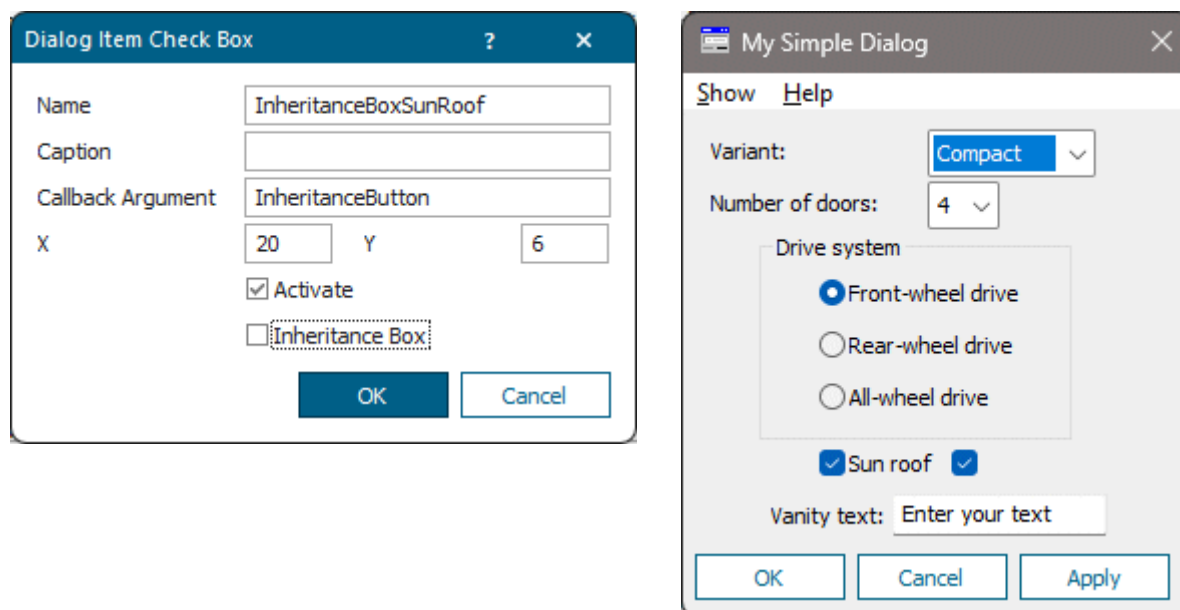
You can specify the following parameters:

- The parameter *DialogItemName* of data type *string* designates the check box for which you want to activate the feature **Inheritance Box**.
- The parameter *Value* of data type *boolean* selects (true) or clears (false) the check box **Inheritance Box**.

Assign true so that the *Dialog* shows the check box as a check box for inheritance.



Assign false so that the *Dialog* shows the check box as a normal check box.



Example

```
MySimpleDialog.setInheritanceBox("InheritanceBoxSunRoof", true)
```

SimTalk

[getInheritanceBox \[SimTalk\]](#)

See also

[New Check Box](#)

setList [SimTalk]

Sets the items of a list which the designated dialog items of the *Dialog* designated by <Path> shows.

Type

Method

Syntax

```
<Path>.setList(DialogItem:string, List:any) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates a **List Box**, a **Drop-down List Box**, or a **Tab Control**.
- The parameter *List* of data type *any* designates the one-columned list into which you enter the **Items** which the dialog items **List Box** or **Drop-down List Box** or the tabs of **Tab Control** show.

Return Value

The return value has the data type *boolean*.

true if the item in the list was addressed and if it changed its display.

Examples

```
MyDialog.setList("sedan", ListSedan)
```

```
var units: list[string]  
units.create  
units.insert(1, "pound")  
units.insert(2, "stone")  
MyDialog.setList("unit", units)
```

See also

[New List Box](#)

[New DropDownList Box](#)

[New Tab Control](#)

Items [Dialog]

setPasswordMasking [SimTalk]

Sets if the check box **Password** of a dialog item of type **Edit Text Box** in the *Dialog* designated by <Path> will be selected or cleared.

Type

Method

Syntax

```
<Path>.setPasswordMasking(DialogItem:string, Selected:boolean) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the check box **Password** ☒ Password.
- The parameter *Selected* of data type *boolean* sets if the check box **Password** will be selected (*true*) or cleared (*false*).

When it is selected the *Dialog* shows text that the user enters into the text box with superscripted lower case xes . When it is cleared it shows the password as clear text .

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.setPasswordMasking("Password", true)
```

See also

[New Edit Text Box](#)

[getPasswordMasking \[SimTalk\]](#)

setSensitive [SimTalk]

Sets if the *Dialog* designated by <Path> shows and activates a dialog item or shows it and dims it.

Remarks

setSensitive applies to all dialog items, except for the **Tab Control**, the **Tab Page** and the **Image**.

Type

Method

Syntax

```
<Path>.setSensitive(DialogItem:string, Activated:boolean) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the dialog item.
- The parameter *Activated* of data *boolean* sets if the dialog item will be shown and activated (*true*) or be shown and unavailable (*false*).

Return Value

The return value has the data type *boolean*.

true if the state of the dialog item is activated and has changed.

false if the state of the dialog item is deactivated, and has not changed.

Example

```
if MyDialog.setSensitive("MyButton", false)
  print "The button 'MyButton' was deactivated."
else
  print "The button 'MyButton' has already been deactivated."
end
```

setTab [SimTalk]

Moves a dialog item onto a **Tab** in the *Dialog* designated by <Path>.

Remarks

Do not use *setTab* for dynamically creating tabs in your dialog!

You cannot move a dialog item within a group with this method.

Type

Method

Syntax

```
<Path>.setTab(DialogItem:string, NewTabPage:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the dialog item.
- The parameter *NewTabPage* of data type *string* designates the **Tab** to which the dialog item is moved.

Return Value

The return value has the data type *boolean*

Example

```
MyDialog.setTab("button", "Properties.Settings")
```

setTable [SimTalk] - Dialog

Sets the table that defines the **Items** which a dialog item of type **List View** shows in the *Dialog* designated by <Path>.

Type

Method

Syntax

```
<Path>.setTable(DialogItem:string, AttributeTable:string/object) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *DialogItem* of data type *string* designates the dialog item of type **List View**.
- The parameter *AttributeTable* of data type *object* or the user-defined variable of data type *string* designated by the parameter *string* designate the attribute table.

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.setTable("list_view", ".Models.Model.Station.ListViewTable")  
MyDialog.setTable("list_view", .Models.Model.ListViewTable)
```

SimTalk

[getTable \[SimTalk\] - Dialog](#)

See also

[Items \[Dialog\]](#)

[New List View](#)

[getTable \[SimTalk\] - Dialog](#)

setTableRow [SimTalk]

Sets the number of the row of the **Items** table which will be selected in a **List View** of the dialog designated by <Path>.

Type

Method

Syntax

```
<Path>.setTableRow(Table:string, Row:integer) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Table* of data type *string* designates the items table.
- The parameter *Row* of data type *integer* designates the row which will be selected.

Return Value

The return value has the data type *boolean*.

Example

```
MyDialog.setTableRow("MyListViewTable",4)
```

SimTalk

[getTableRow \[SimTalk\]](#)

See also

[Items \[Dialog\]](#)

[New List View](#)

updateUserDialog [SimTalk]

Updates the contents of the *Dialog* designated by <Path>.

Remarks

The *Dialog* then shows features that you changed in tables defining the items, or shows new or changed pictures and icons of the items.

Type

Method

Syntax

```
<Path>.updateUserDialog
```

Example

```
MyDialog.updateUserDialog
```

Read-Only Attributes of the Dialog

Read-Only Attributes of the Dialog

The *Dialog* provides the [_Read-Only Attributes of All Objects](#).

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.